

Class #13

04.03 Applied Architecture Patterns

Software Architectures · Master in Informatics Engineering
Reuse, sidecars, workflow ownership, and production recovery

Agenda

- Opening - from coupling to applied patterns 10 min
- Block 1 - ADR 13.1: shared library, shared service, or duplication? 35 min
- Block 2 - ADR 13.2: operational sidecar and service mesh 45 min
- Block 3 - ADR 13.3: primary ticket workflow saga 60 min
- Block 4 - production issue walkthrough 45 min
- References - pattern catalogues and decision sources references

The filter for today

Do not ask first:

Can we use this pattern?

Ask: what coupling, failure, or ownership problem would justify it?

Each major choice is framed as an Architecture Decision Record (ADR).

Old topic, larger scale

In the first half of the semester, coupling meant reasons to change inside code, modules, and boundaries.

Today, the same question appears at a larger scale.

Who changes together?

Who fails together?

Who must coordinate?

Who owns recovery?

Where we are in the story

We already looked at architecture decisions, data ownership, reliable events, duplicate handling, and dependency failure.

Today: shared behavior, workflow ownership, and production recovery.

Three practical questions:

- which coupling are we accepting when we reuse?
- where should repeated operational behavior live?
- who owns a workflow after it crosses service boundaries?

Reuse is not free

Reuse is coupling with a better reputation.

- copied code drifts
- shared libraries coordinate versions
- shared services add runtime coupling
- platforms add operational complexity

ADR 13.1

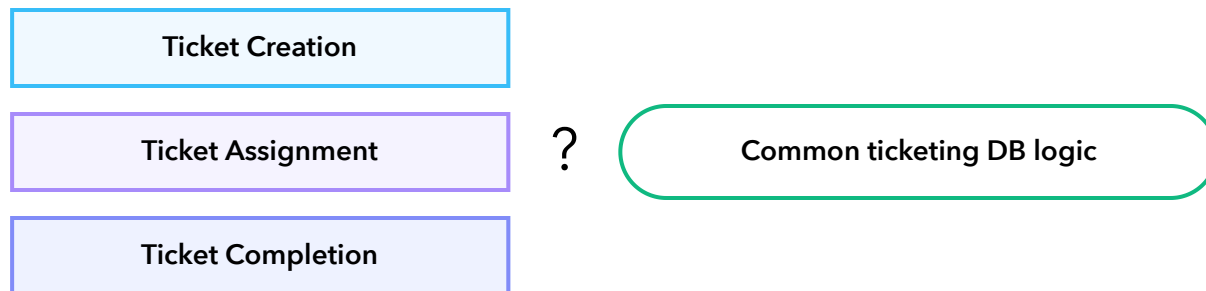
Shared library, shared service, or duplication?

Some reuse decisions are architecture decisions in disguise.

Reference: *Hard Parts*, Appendix B, "Use of a Shared Library for Common Ticketing Database Logic".

1. Context

Ticket Creation, Ticket Assignment, and Ticket Completion share common ticketing database logic.



Question: if several services need the same behavior, which kind of coupling do we accept?

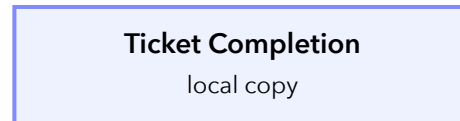
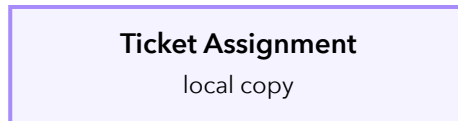
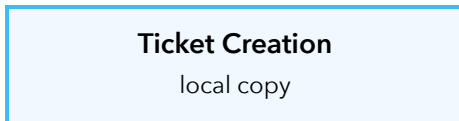
2. Proposed solutions

Option	What it tries to preserve
Code replication	independent change
Shared library	one implementation at build time
Shared service	one implementation at runtime
Domain-owned service	explicit business ownership

Reuse Strategy

Same behavior, different coupling

Option 1: code replication

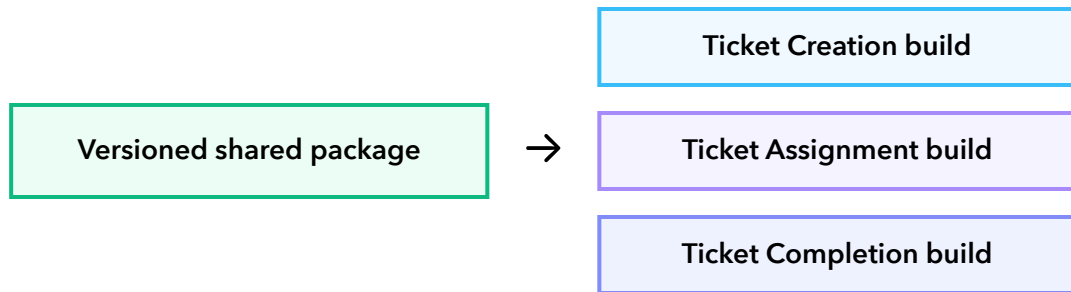


Mechanism: each service keeps its own implementation of the repeated behavior.

What to inspect:

- how often the behavior changes
- whether different versions can be tolerated
- who notices drift

Option 2: shared library

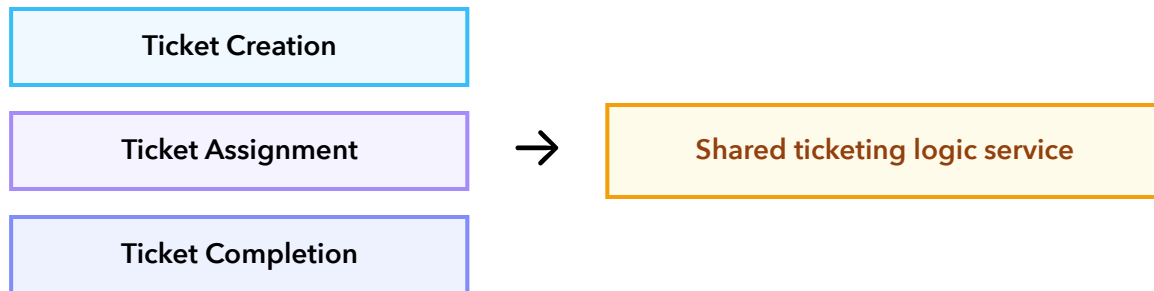


Mechanism: common behavior ships as a dependency included by each service.

What to inspect:

- language/runtime alignment
- versioning discipline
- redeploy and test cost when the library changes

Option 3: shared service

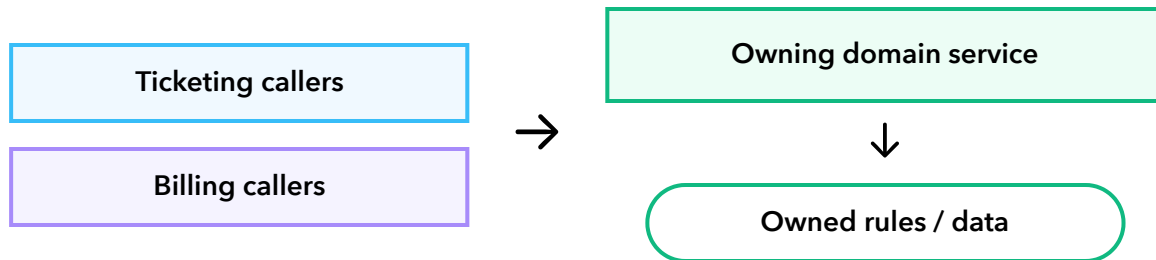


Mechanism: callers ask one runtime service for the shared behavior.

What to inspect:

- latency and availability on the caller path
- ownership of the service contract
- scaling and failure isolation

Option 4: domain-owned service



Mechanism: the domain that owns the behavior exposes it through an explicit service contract.

What to inspect:

- whether the behavior is business-significant
- whether one domain can honestly own the rule
- whether callers are asking the right domain question

3. Technical analysis

Option	Works because	May fail because
Code replication	independent teams can move without coordination	silent drift can become product inconsistency
Shared library	no runtime dependency and one implementation	build/version coupling and coordinated redeploys
Shared service	one live owner for shared behavior	every caller inherits latency and availability coupling
Domain-owned service	ownership is explicit and visible	the service can become a generic dependency for unrelated concerns

Evaluation forces

For ADR 13.1, the dominant forces are:

- change frequency
- deployability
- runtime coupling
- team ownership
- performance and fault tolerance
- supportability

Accepted damage depends on the coupling we choose: drift, version coordination, or runtime dependency.

What's the verdict and why?

The book ADR chooses a shared library for common ticketing database logic.

Reason: the shared logic changes infrequently, can be versioned, and avoids adding a runtime service dependency to the customer-facing ticket path.

Recap: the flaw is not duplication itself; the flaw is unmanaged coupling.

ADR 13.2

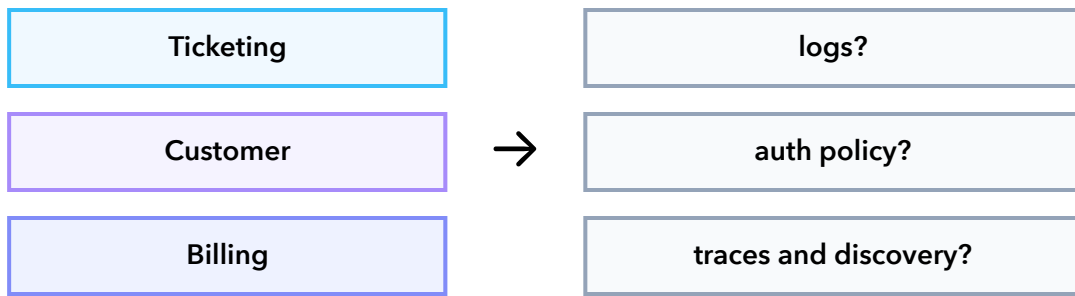
Sysops Squad operational adapter sidecar

Where should common operational behavior live?

References: *Hard Parts* Appendix B ADR "Using a Sidecar for Operational Coupling"; Azure Sidecar.

1. Context

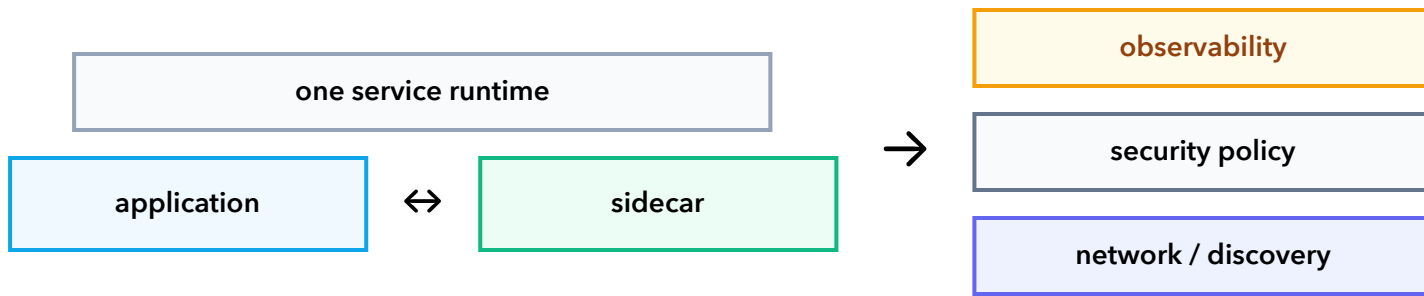
Every service needs common operational behavior. Leaving it to each team creates inconsistency and coordination cost.



The source ADR lists monitoring, logging, service discovery, authentication, and authorization as shared sidecar concerns.

What is a sidecar?

A sidecar is a companion process or container deployed beside an application instance.



The sidecar should carry shared operational behavior, not business ownership.

Putting a sidecar in play

The application and sidecar are deployed as one runtime unit. The sidecar can handle telemetry and edge policy before the request reaches application code.

```
apiVersion: v1
kind: Pod
metadata:
  name: ticketing
spec:
  containers:
    - name: ticketing-api
      image: sysops/ticketing-api:1.8
      ports:
        - containerPort: 8080

    - name: ops-sidecar
      image: sysops/ops-sidecar:2.1
      ports:
        - containerPort: 15001
      env:
        - name: APP_UPSTREAM
          value: http://127.0.0.1:8080
        - name: OTEL_EXPORTER_OTLP_ENDPOINT
          value: http://collector:4317
        - name: JWT_ISSUER
          value: https://auth.sysops.local
        - name: AUTHZ_POLICY
          value: ticketing-ingress-v3
```

What changed?

- the ticketing application still owns ticketing logic
- the sidecar forwards traces and logs
- the sidecar can validate tokens or enforce mTLS policy
- deployment now includes an application lifecycle and a sidecar lifecycle
- troubleshooting must inspect both processes

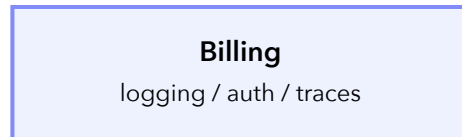
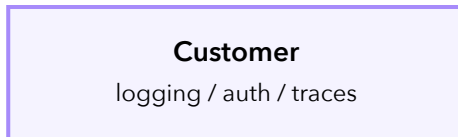
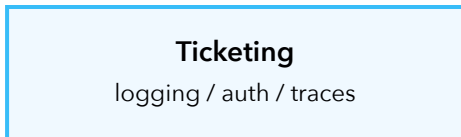
2. Proposed solutions

Option	What it tries to preserve
Each team implements operational behavior	service autonomy
Shared operational library	code reuse
Shared operational service	one runtime owner
Sidecar per service instance	local operational adapter
Full service mesh	platform-level traffic/security control

Operational Reuse

Shared behavior near the service, not inside the domain

Option 1: each service implements it

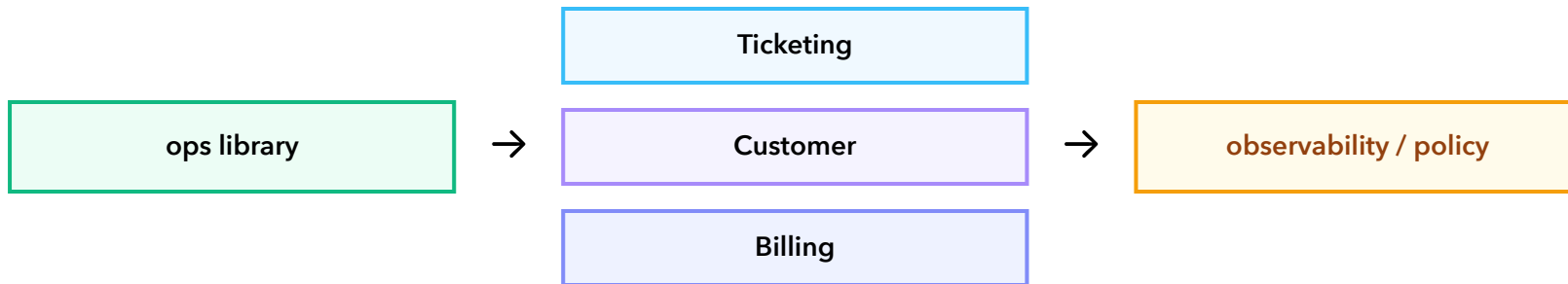


Mechanism: each team codes the operational behavior inside its own service.

What to inspect:

- how many services need the behavior
- how quickly the operational standard changes
- whether inconsistency is acceptable

Option 2: shared operational library

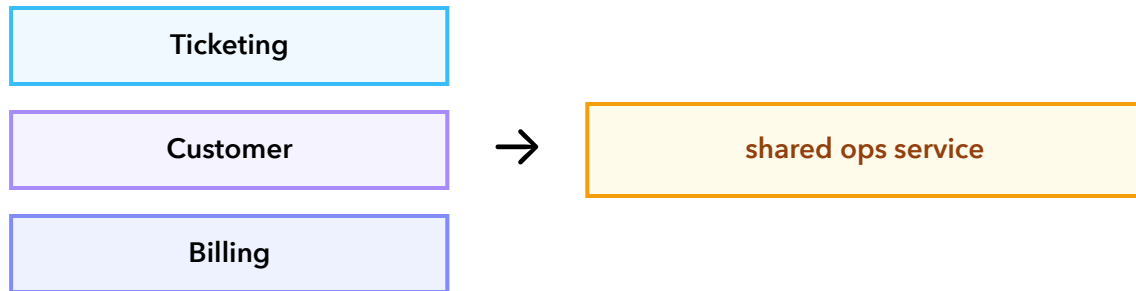


Mechanism: package common operational behavior as a dependency.

What to inspect:

- language and runtime variety
- version drift
- upgrade discipline

Option 3: shared operational service

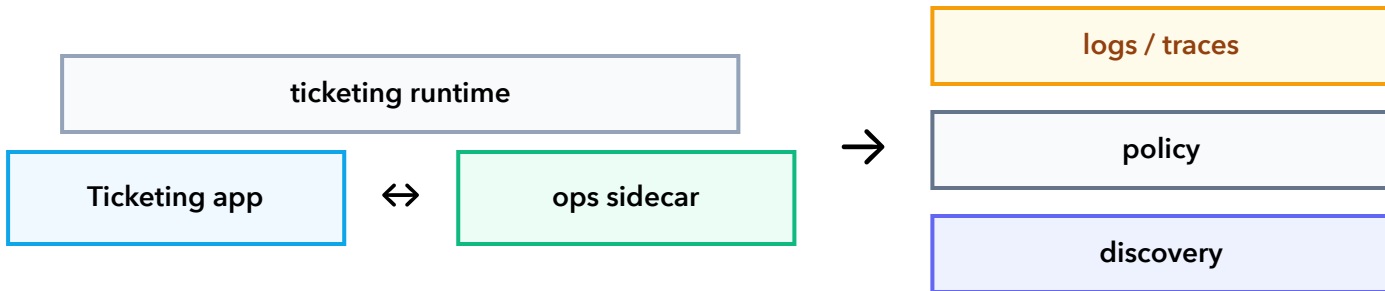


Mechanism: route common operational calls through a central service.

What to inspect:

- whether request paths now depend on this service
- latency tolerance
- failure behavior when the shared service is unavailable

Option 4: sidecar per service

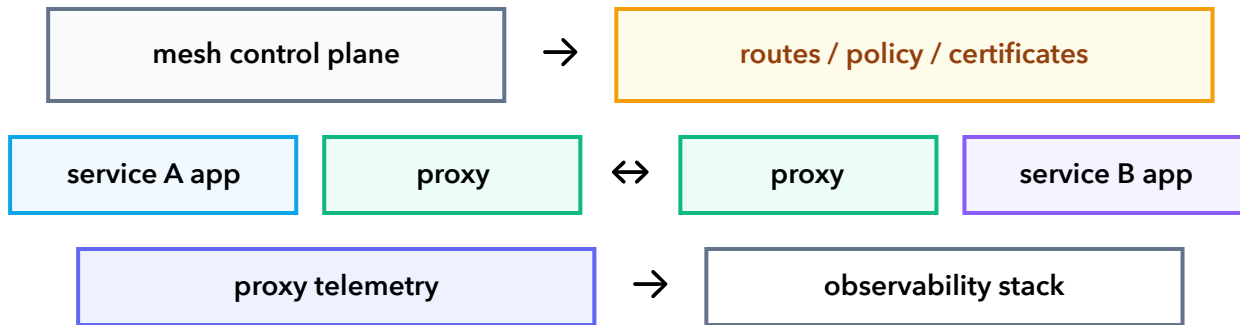


Mechanism: deploy an operational companion beside each service instance.

What to inspect:

- deployment and ownership of the sidecar
- local resource cost
- troubleshooting across application and sidecar

Option 5: full service mesh



Mechanism: use sidecar proxies plus a control plane for traffic and security policy.

What to inspect:

- service estate size
- platform maturity
- operational ownership of the mesh itself

Mesh is more than one proxy

application services

domain behavior

data plane proxies

traffic, mTLS, retries, telemetry

control plane

configuration and policy

A mesh moves operational policy into the platform. That is useful only if the platform team can own the extra machinery.

Mesh policy in play

One control-plane rule can affect many service-to-service calls.

```
traffic:
  from: ticketing
  to: billing-verifier
  tls: required
  timeout: 800ms
  retries:
    attempts: 2
    retryOn: 5xx,connect-failure

authorization:
  allow:
    - source: ticketing
      path: /verify-billing
      method: POST
```

What changed?

- service code does not implement every network rule
- proxies enforce traffic and auth policy on the path
- platform configuration now affects runtime behavior
- a bad policy can break many services at once

3. Technical analysis

Option	Works because	May fail because
Per service	high autonomy	operational behavior becomes inconsistent
Shared library	familiar reuse	version and language coupling
Shared service	one runtime owner	runtime dependency appears in operational paths
Sidecar	local adapter with less app-code change	lifecycle and troubleshooting complexity
Service mesh	strong platform control	high operational burden

Evaluation forces

For ADR 13.2, the dominant forces are:

- operational consistency
- team ownership
- supportability
- deployability
- runtime coupling

Accepted damage: platform/runtime complexity and app + sidecar troubleshooting.

What's the verdict and why?

Use sidecars for common operational capabilities only.

Good sidecar material: telemetry forwarding, discovery adapter, mTLS/auth policy, traffic policy.

Not sidecar material: ticket assignment rules, customer workflow, billing policy, or data ownership.

Recap: the problem was uneven operational behavior, not shared domain logic.

ADR 13.3

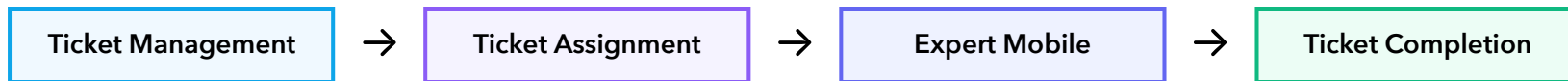
Primary ticket workflow saga

Who owns a workflow that crosses service boundaries and can fail halfway?

Public references: [Microservices.io Saga](#), [Azure Saga](#).

1. Context

A customer submits a trouble ticket and receives a ticket number. Work continues after that first visible response.



Question: once the workflow is no longer inside one service, where does workflow ownership live?

What is a saga?

A saga is a sequence of local transactions coordinated to complete a larger business workflow.

Each step commits locally. If a later step fails, the system needs a defined recovery action.

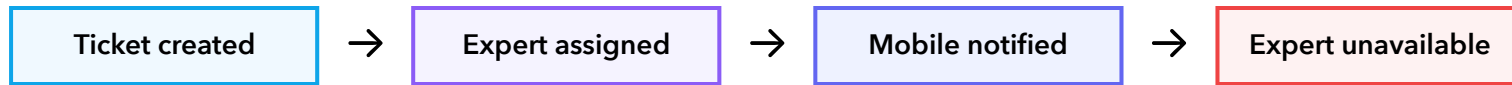
A saga is not one distributed transaction with a nicer name.

Saga vs distributed transaction

Question	Distributed transaction	Saga
Commit model	one coordinated atomic commit	several local commits
Failure handling	rollback before the commit finishes	retry, compensation, escalation, or cancellation after partial progress
State owner	transaction coordinator and infrastructure	workflow owner or participating services
Business fit	tries to make the whole action all-or-nothing	accepts that some actions are durable business facts

A saga is not stronger atomicity. It is explicit recovery after local commits.

Concrete failure path



The hard question: wait, retry, reassign, compensate, escalate, or cancel?

What must be visible?

For this workflow, hidden failure is worse than visible delay.



If support cannot see the state, the architecture has pushed the problem onto people.

2. Proposed solutions

Option	What it tries to preserve
One service boundary	local workflow control
Local transaction + async follow-up events	simple owner, delayed work
Event choreography	autonomous services
Orchestrated saga with workflow state	visible process ownership
Manual support recovery	human judgement for rare failures

Workflow Consistency

Compensation is not undo

Option 1: one service boundary

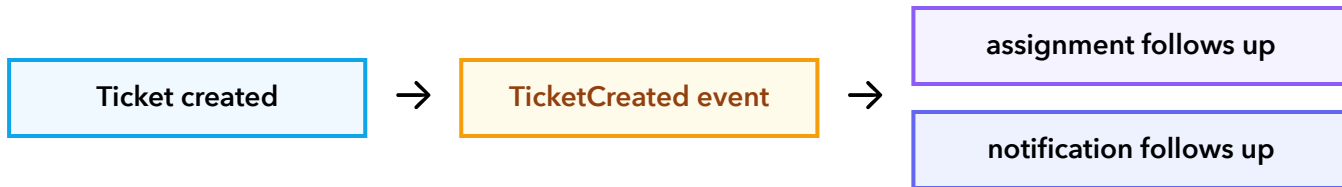


Mechanism: keep the primary workflow inside one service boundary.

What to inspect:

- whether the workflow is truly one business capability
- whether the boundary is hiding unrelated ownership
- how often parts of the workflow change independently

Option 2: local transaction + async follow-up

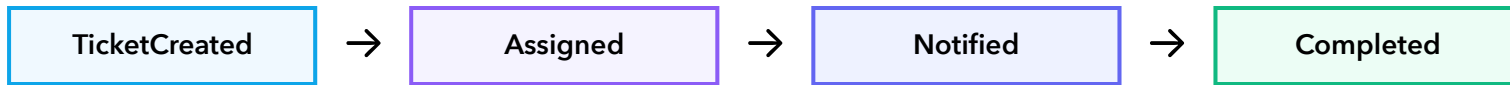


Mechanism: commit the local ticket change, then let other services react later.

What to inspect:

- whether follow-up can lag
- whether follow-up failure blocks the business promise
- how visible delayed work is

Option 3: event choreography

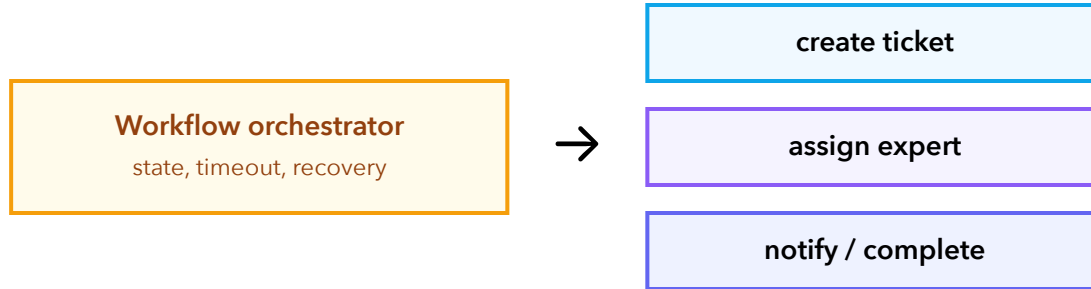


Mechanism: each participant reacts to events and emits the next fact.

What to inspect:

- where the current workflow state can be queried
- who owns timeouts and recovery
- what happens when an event is late or duplicated

Option 4: orchestrated saga



Mechanism: a process owner sends commands, records workflow state, handles timeouts, and chooses recovery actions.

What to inspect:

- whether support visibility dominates
- whether recovery actions are explicit
- whether the orchestrator becomes the constraint

Option 5: manual support recovery



Mechanism: detect the stuck workflow, create a support case, and let an operator correct or escalate.

What to inspect:

- how frequent this path is
- whether the stuck state is visible
- whether the operator has enough evidence to act

3. Technical analysis

Option	Works because	May fail because
One boundary	local workflow state and simple recovery	may hide a real ownership split
Local + async	simple and resilient enough for follow-up	weak coordinated completion
Choreography	autonomy, no central controller	poor process visibility
Orchestrated saga	visible workflow state and recovery	process-manager coupling
Manual recovery	handles rare ambiguity	unacceptable for common flows

Exercise

Build the evaluation forces matrix for ADR 13.3

20 min

Compare the options against the forces before choosing.

Matrix skeleton

Option	Deployability	Consistency	Operations	Fault isolation	Ownership	Supportability	Reversibility
One service boundary	medium	high local	low	low/medium	concentrated	high if simple	medium
Local + async follow-up							
Event choreography							
Orchestrated saga							
Manual recovery							

The value is in the explanation: which forces matter, which damage you accept, and why that trade-off fits this workflow.

Evaluation forces

For ADR 13.3, the dominant forces are:

- workflow visibility
- error handling
- recoverability
- scalability
- service decoupling
- supportability

Accepted damage: process-manager coupling, idempotency burden, and compensation design.

Compensation is a design object

For the concrete failure path:

- ticket creation may be durable
- expert assignment may need reassignment
- notification may require correction, not undo
- delayed completion may require escalation

Saga design is the explicit naming of those states and actions.

What's the verdict and why?

Use a saga only if the workflow truly crosses service boundaries and needs coordinated completion or recovery.

Prefer orchestration when support visibility, timeouts, and explicit recovery matter.

Prefer local + async follow-up when later work can fail independently.

Recap: the problem was workflow ownership under partial failure, not replacing a database transaction with a diagram.

Production Issue Walkthrough

The stuck ticket workflow

A customer has a ticket number. Support cannot tell whether the work is progressing.

The production report

At 09:47, support opens an incident.

Customers can create tickets, but some tickets remain without expert action for more than 40 minutes.

Known observations:

- ticket number returned to customer
- confirmation sometimes duplicated
- billing setup is slow

Unknowns:

- was an expert assigned?
- was the mobile app notified?
- is the workflow stuck or just delayed?

Normal path



First, follow the evidence.

Timeline seen by support

Time	Observation
09:02	customer receives ticket number T-48291
09:03	confirmation email sent
09:04	billing setup begins
09:06	confirmation email sent again
09:12	billing setup retry begins
09:42	no expert action visible

What do we need to know before proposing a fix?

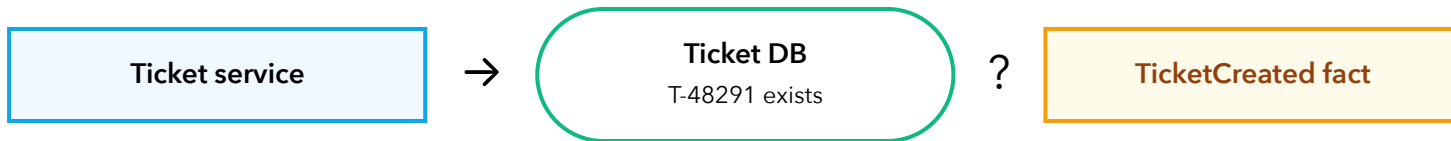
First question

Which part of the architecture can answer:

Where is ticket T-48291 now?

Evidence path A

The ticket was created



The local state exists. The next question is whether the fact left the service reliably.

Evidence path B

The event may have been published more than once



The event pipeline is active. The repeated effect needs a separate explanation.

Evidence path C

Billing is slow



Retrying might help. Retrying might also make the incident worse.

Evidence path D

The expert assignment is not visible



If there is no visible workflow state, support only sees fragments.

Diagnosis pause

For each symptom, answer two questions:

1. What evidence can show it?

Logs, traces, local state, message identity, workflow state, or dependency metrics.

2. What mechanism can recover it?

Retry, idempotency, outbox relay, circuit breaker, reassignment, compensation, or escalation.

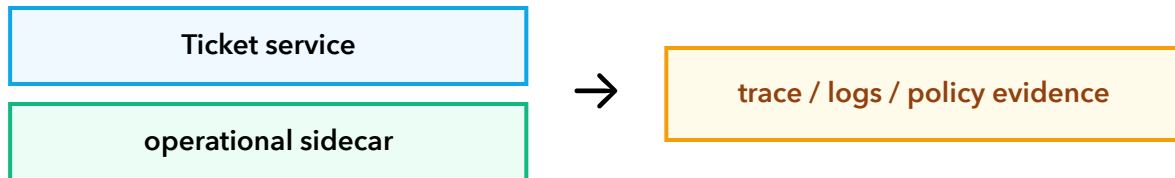
Diagnostic table

Symptom	Evidence needed	Candidate mechanism
Ticket exists locally	committed ticket state and publish trace	
Confirmation sent twice	message identity and consumer history	
Billing is slow	dependency latency, retry count, breaker state	
Expert action missing	workflow state, timeout, assignment state	
Support sees fragments	correlated trace and workflow record	

Keep the last column blank until the evidence is clear.

What lets us see the path?

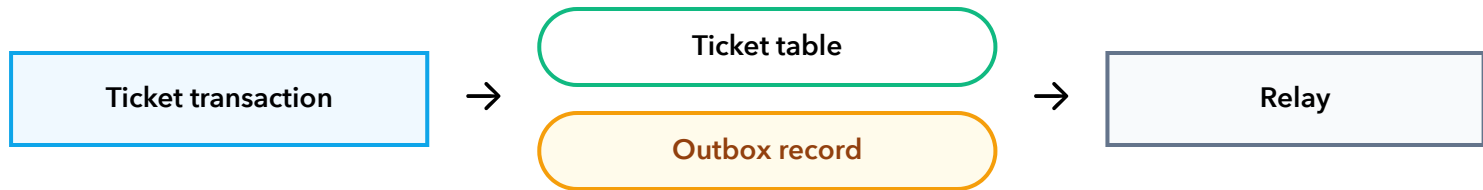
Operational telemetry and policy evidence



This does not fix the workflow. It makes the path inspectable.

What proves the committed fact left the service?

Reliable publication



This protects publication. It does not prevent duplicate handling downstream.

What prevents the duplicate effect?

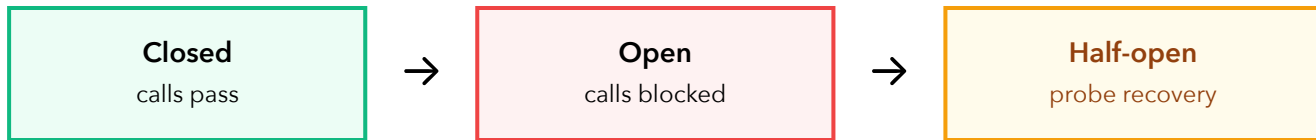
Idempotent consumption



This protects the destination effect. It does not decide the whole workflow.

What stops billing from consuming the caller?

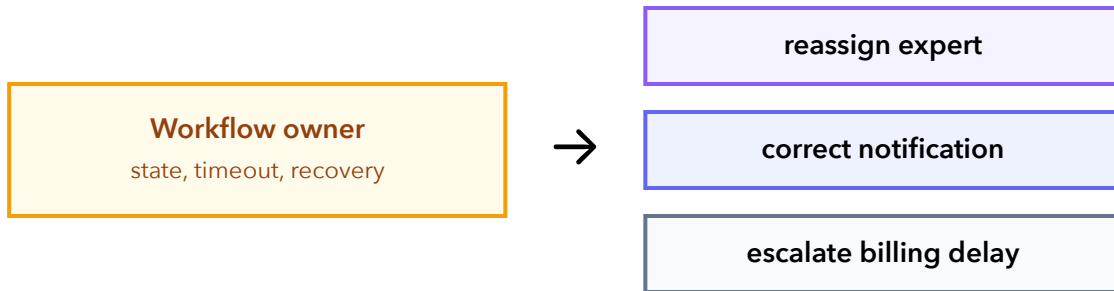
Dependency protection



This contains dependency pressure. It still needs a business answer for degraded behavior.

Who owns recovery?

Workflow state and recovery



This owns the workflow. It does not replace the mechanisms that make facts reliable.

Filled diagnostic table

Symptom	Evidence needed	Mechanism that helps
Ticket exists locally	committed ticket state and publish trace	outbox + trace
Confirmation sent twice	message identity and consumer history	inbox / idempotent consumer
Billing is slow	dependency latency, retry count, breaker state	timeout + bounded retry + circuit breaker
Expert action missing	workflow state, timeout, assignment state	saga / workflow owner
Support sees fragments	correlated trace and workflow record	sidecar telemetry + workflow state

No single pattern explains the incident.

Production fix shape

Immediate containment:

- stop duplicate side effects
- expose ticket workflow state
- cap retries to billing

Architecture follow-up:

- make publication recoverable
- make consumers idempotent
- define workflow owner and recovery actions

The incident is resolved when the system can explain the ticket, not just when the ticket eventually completes.

Final Architecture Judgement

What each pattern does not solve

Pattern boundaries

Mechanism	Helps with	Does not solve
Sidecar	operational visibility and policy consistency	workflow ownership
Outbox	reliable publication of committed facts	duplicate side effects
Inbox / idempotent consumer	duplicate-safe consumption	event ordering or workflow recovery
Circuit breaker	dependency pressure and caller protection	business compensation
Saga / workflow owner	visible long-running recovery	local atomicity or message reliability

Patterns compose, but their guarantees do not merge automatically.

The last filter

Before keeping a pattern, ask:

What failure does it make visible?

What failure does it contain?

What recovery action does it enable?

What new burden does it create?

If a pattern cannot name the failure it handles, it is decoration.

References

Use after the failure is clear

References and what they are good for

Source	Good for
<i>Software Architecture: The Hard Parts</i>	ADR examples, trade-off analysis, reuse, operational coupling, workflow ownership, sagas
<u>Azure Sidecar pattern</u>	sidecar intent, deployment shape, benefits, and cautions
<u>Azure cloud design patterns</u>	retry, circuit breaker, health endpoint monitoring, sidecar, saga, queue-based load leveling
<u>Microservices.io Saga</u>	saga structure, choreography vs orchestration, and database-per-service consistency pressure
<u>Microservices.io Transactional Outbox</u>	reliable publication after a local commit
<u>Microservices.io Idempotent Consumer</u>	duplicate-safe message handling

Read catalogues as decision support, not as shopping lists.